

RESEARCH ARTICLE

Real-Time Sensing and On-Site Spotting Scheme of Multi-Type WLAN Spycams

HYEYOUNG AN^{ID}, WOOJIN PARK^{ID}, (Student Member, IEEE),
AND SOOCHANG PARK^{ID}, (Member, IEEE)

Department of Computer Engineering, Chungbuk National University, Cheongju, Republic of Korea

Corresponding author: Soochang Park (cewinter@chungbuk.ac.kr)

ABSTRACT This paper proposes a real-time sensing and on-site spotting scheme for multi-type wireless LAN based spy-cameras. The proposed scheme has the goal of overcoming the restrictions of previous studies on location-dependent training data per target location for spycam traffic classification and learning-based positioning, and the overhead requiring additional equipment to gather specific data for spycam location such as channel state information (CSI). In addition, some ways to send video data via spycams exploit data encryption, so it leads to erasing feature of video data which should be trained to the models for camera existence detection. In other words, there is a lack of comprehensive research that integrates the detection and localization of these surveillance video devices without reliance on data preparation per location and special devices. Recognizing the need to detect video devices used for such malicious purposes in real-time, this paper introduces a method that utilizes the Nilsimsa hashing algorithm and the Log-Distance Path Loss (LDPL) model for similarity checks to detect and localize various video devices transmitting wirelessly. We capture the wireless data transmitted and use the Nilsimsa hashing algorithm, which generates similar hash results for similar inputs, to detect video devices through similarity checks. Furthermore, we rely only on the recent signal strength indication (RSSI) values received to quickly estimate the location of the device that is not very accurate but detectable with eyes at the site, regardless of its type. Our approach shows determining the presence of a device within 3 seconds and estimating its approximate location with an average accuracy of approximately 0.08 m within a 1 m range and 0.45 m within a 3 m range, regardless of the device type.

INDEX TERMS IP camera, drone, unmanned aerial vehicles (UAV), Nilsimsa, Wi-Fi received signal strength indication (RSSI), traffic analysis, log-distance path loss (LDPL).

I. INTRODUCTION

The convergence of state-of-the-art technology has led to the widespread use of diverse surveillance devices in various scenarios. Especially, IP cameras and drones are prevalent, offering customized solutions to users in terms of affordability, user-friendliness, and adaptability to diverse environments [1], [2], [3], [4]. The stationary IP cameras are typically installed in fixed positions to meet the demand for tracking or monitoring the movements of parents, children, pets, and the like [3] and [4]. On the other hand, unlike stationary IP cameras, mobile cameras such as drones offer particularly

advantageous customized solutions in scenarios related to emergencies, complex terrains, and limited transportation infrastructure, as they boast exceptional mobility and can quickly access almost any location [4], [5]. Moreover, drones' controlled mobility facilitates enhanced data collection in remote areas of wide-ranging interest and reduces collection costs by establishing air-land environmental monitoring systems [6]. This allows drones to be applied in various fields, from surveillance and monitoring to delivery services and risk management [1], [2].

However, despite their usefulness, both stationary and mobile cameras, which capture and transmit video footage, can easily be exploited for malicious purposes. IP cameras (spy cameras) installed for malicious purposes are being

The associate editor coordinating the review of this manuscript and approving it for publication was Guillermo Valencia-Palomo^{ID}.

used to secretly capture private moments in settings such as bathrooms, dressing rooms, and hotel rooms without someone's permission, with the footage being sold on various illicit websites [7]. Additionally, drones are exploited not only for malicious purposes such as surveillance and reconnaissance for bombing and warfare but also for nefarious activities like drug trafficking and smuggling, raising significant international concerns [1], [5]. These concerns underscore the necessity for focused research efforts on detecting surveillance equipment exploited for malicious purposes. Distinguishing between devices used for legitimate purposes and those exploited for illegal activities is crucial to safeguarding public safety and protecting individual privacy.

However, as technology advances, devices have become smaller, lighter, and more powerful. Consequently, their complex nature makes it difficult to accurately detect, identify, and recognize them. Therefore, it is essential to thoroughly analyze detection methodologies that take into account the characteristics of surveillance equipment. These devices leverage a range of sensor technologies, such as GPS (Global Positioning System), LiDAR (Light Detection and Ranging), RaDaR (Radio Detection and Ranging), cameras, and various information processing and communication technologies [1]. The high-resolution capabilities of the camera enable the collection of video data from remote areas of interest [2]. Additionally, wireless and internet connectivity facilitates the transmission of control commands and these video streams [2], [6]. It is important to remember that video surveillance equipment primarily utilizes infrastructure-based networks such as Ethernet, WLAN (wireless LAN), and cellular networks for video data transmission [5]. Among these, Wi-Fi-based approaches are the most promising of these technologies in terms of versatility and convenience in various circumstances [8].

Accordingly, network traffic-based detection methods are commonly employed to detect these surveillance devices. These methods are sometimes applied individually or combined with machine learning (ML) techniques or other methodologies for enhanced accuracy and efficiency [3]. Spy cameras are identified using specialized hardware, CSI analysis tools, or traffic comparison methods. Using hardware devices to detect spy cameras by analyzing network traffic has limitations, such as being impractical due to the device's portability, and being able to detect only specific types of streams. CSI analysis methods also require dedicated devices for analysis and may struggle to detect spaces without movement.

Compared to IP cameras, drone detection technologies encompass a broader range of methods beyond traffic analysis, including the use of infrared cameras to detect heat signatures, sensors that monitor RF signals and radar, visual methods using cameras, and sound detection techniques [9]. However, with the increase in noiseless drones, sound detection techniques often fail to detect drones accurately. Additionally, visual technologies are greatly influenced by

lighting and weather conditions. Furthermore, radar-based approaches are frequently affected by fog, clouds, and dust, leading to decreased detection accuracy [10].

Network traffic-based detection methods are commonly employed for drones and IP cameras. However, traditional approaches primarily involve comparing traffic signals or using ML techniques. ML methods require extensive data collection to produce effective results. Additionally, if the detection location changes and the ML model has not been trained on the new location, the accuracy may decrease.

Therefore, a simplified approach that does not require specialized equipment, learning complex detection techniques, or large amounts of data is necessary. Our approach ensures consistent, practical, reliable detection across various locations and conditions. To achieve this, our method introduces a traffic-based mechanism that identifies wireless streaming surveillance devices near the user in real-time, utilizing simple setups and minimal computational resources. As a result, our method presents a practical and consistent approach to detecting devices transmitting data wirelessly in real-time, regardless of the type of device.

In addition to detection, effective localization is equally important for accurately identifying the position of these devices. Device localization techniques primarily utilize signal characteristics, often relying on methods such as Round-Trip Time (RTT), CSI, and RSSI [11], [12]. However, both RTT and CSI typically necessitate specialized equipment or rely on human movement for effective detection. Therefore, we utilize RSSI, which is the most widely adopted method for indoor localization due to its high accessibility on most smart devices [13]. One of the advantages of using RSSI is that it allows us to leverage the same packets utilized in detection, eliminating the need for additional equipment. This enables both detection and localization to occur simultaneously, enhancing the overall efficiency of our approach.

We summarize our main contributions as follows:

- 1) **Concurrent identification and localization:** When collecting packets, not only are encrypted packets captured, but specific packet features (e.g., RSSI, source address, destination address) are also extracted for analysis. The collected packets are subjected to the Nilsimsa algorithm without any preprocessing to evaluate packet similarity for device detection, while the RSSI values are used to simultaneously determine the device's location. Consequently, both detection and localization are achieved within 3 seconds.
- 2) **Unified Method Regardless of Device Type:** We apply a consistent identification and distance estimation method for surveillance video devices, whether stationary or mobile, through our packet analysis approach. For identification, we use the Nilsimsa algorithm to evaluate packet similarity, while during localization, the RSSI values applied using the LDPL model are estimated using a uniform variable value

TABLE 1. Comparison of techniques used in surveillance devices detection.

Device	Detection method	Strengths	Weaknesses	Ref.
IP Camera	CSI	- High detection accuracy - Sensitivity to changes in radio waves - Effective indication of human presence	- Requires dedicated devices - Requires specialized knowledge for CSI data analysis	[7], [16], [11], [12]
	Traffic pattern analysis	- Analyze MAC addresses without encryption or protocol knowledge - Detect Wi-Fi-enabled surveillance devices	- Only detect live streaming spy cameras on Wi-Fi networks - Unable to detect devices transmitting stored video at a later time	[17], [18]
	ML + Traffic pattern analysis	- Improved detection using trained models compared to traditional traffic analysis - Enhanced accuracy and robustness in detecting IP cameras	- Difficulty in obtaining large-scale labeled data for model improvement - Requires significant computing resources and expert knowledge - Performance degradation due to environmental changes	[19]–[21]
Drone	Passive RF detection method	- Detects drones even without active data transmission or non-standard protocols - Enables effective detection over a broader range	- Requires expensive and complex RF receivers and signal processing equipment - Accuracy is reduced due to RF signal interference from other devices	[23]
	Wi-Fi traffic pattern analysis	- Utilizes existing Wi-Fi infrastructure, reducing the need for additional equipment - Minimizes the risk of being detected or interfering with the drone	- Limited ability to detect drones using non-Wi-Fi communication protocols - Performance varies with network traffic, signal interference, or environmental conditions	[1], [24]–[28]
IP camera + Drone (Our method)	Traffic pattern analysis	- Reliable identification and distance estimation for both stationary and mobile surveillance devices - Enables real-time detection through packet analysis without the need for specialized equipment or extensive data collection		

rather than being tailored to each device. This approach eliminates the need for device-specific detection methods and extensive location-specific data collection and training.

- 3) **Simplified and reliable approach:** Our approach leverages commonly available commercial devices equipped with Wi-Fi monitoring capabilities, relying solely on packet analysis. This ensures reliable and feasible detection without the need for specialized equipment, complex detection methods, or large volumes of data.

The remainder of this paper is organized as follows: Section II discusses related work. Section III describes the basic packet structure required for packet capture on wireless networks, the method for selecting necessary values from packets, and the packet collection and classification. Additionally, it provides insights into detecting streaming devices in wireless networks using the Nilsimsa algorithm and packet comparison algorithms, along with elucidating the approach to locating these devices through the LDPL model. Section IV discusses the experimental setup and presents the results, along with a discussion. Finally, Section V concludes the paper and outlines future research directions.

II. RELATED WORK

There are two main challenges in detecting malicious surveillance devices. First, it is essential to understand the key characteristics of these devices and classify them to distinguish them from other existing devices, which is

directly related to detection. Second, it is necessary to determine whether these devices are capturing footage in the vicinity without consent, which pertains to localization.

One of the primary characteristics of surveillance video systems is their ability to capture and transmit video footage. IP cameras typically transmit video data via LAN (Local Area Network) connections and Wi-Fi. However, IP cameras used for malicious purposes frequently rely on wireless data transmission [3].

In contrast, drones, which provide mobility for outdoor monitoring, utilize various communication methods, including Wi-Fi, Bluetooth, satellites, and cellular networks [5]. Using cellular communication frequencies like 3GHz, 4GHz, 4G+ (LTE), and 5GHz for drone-network infrastructure communication guarantees stable throughput [5]. Despite these advantages, the limited uplink capacity restricts system performance and increases network costs. Additionally, Cellular has much higher inter-drone communication latency compared to Wi-Fi [14]. Satellite communications provide security and reliability and are well-suited for mission-critical operations, but they are expensive and have demanding maintenance requirements [5].

Wi-Fi Direct, which is based on the IEEE 802.11 standard, allows devices to connect more widely and at higher bandwidths without requiring infrastructure like base stations (BS) or access points (AP) [15]. Wi-Fi Direct establishes a private Wi-Fi network that users can access by entering a password, allowing them to control the drone using a dedicated remote controller, smartphone, or tablet [1]. Wi-Fi Direct

provides superior coverage and transmission rates compared to Bluetooth, utilizing Device-to-Device communication technology. This makes it particularly useful for scenarios where infrastructure-based networks are unavailable, such as during natural disasters or in congested areas [15].

Given that IP cameras and drones commonly utilize Wi-Fi channels for communication, leveraging these common communication protocols for detection can simplify device identification and enhance the efficiency of surveillance device management. This approach ensures a unified and effective detection strategy for various surveillance environments.

In this context, there is extensive research on analyzing communication traffic to detect these video surveillance devices. As shown in Table 1, studies focused on analyzing the traffic patterns of IP cameras predominantly employ methods such as CSI, traffic pattern analysis, and ML for identifying IP cameras [3]. These methods can be applied individually for detection, but they are often combined to improve detection accuracy and reliability.

CSI provides detailed insights into the wireless communication channel between the transmitter and receiver [7], [11]. CSI can offer high detection accuracy due to the unique information it provides. CSI-based detection techniques are highly sensitive to variations in radio frequency propagation, making them effective for detecting changes caused by human presence, and thus widely adopted in various systems [12]. However, this sensitivity also poses a limitation, as device detection becomes challenging in the absence of movement [3]. There is no standardized method for capturing CSI data, and its analysis and interpretation require specialized knowledge. Furthermore, the process often necessitates the use of specialized hardware or software [12].

Conventional traffic pattern analysis methods examine the MAC header, which contains essential information such as source and destination addresses, to analyze network traffic generated by IP cameras based on characteristics such as packet size, transmission intervals, and data flow patterns [17]. Since IP cameras continuously stream video, they often exhibit consistent traffic patterns. SCamF reconstructs video frames using packet length, direction, and sequence number, and detects based on frame size changes due to human motion and the frame size pattern between the user and the camera [17]. However, this method can be affected by the movements of people in the camera's field of view or changes in the environment. SnoopDog detects cameras based on a significant increase in transmission rate when the laptop screen is directly facing the camera lens, yet specific tasks or environmental factors may influence its accuracy [18].

ML further enhances detection capabilities by training models on the distinctive traffic patterns of IP cameras. These models can learn to differentiate IP camera traffic from other devices by analyzing various features extracted from the network traffic [19], [20], [21]. Integrating ML with traffic pattern analysis holds significant potential for enhancing the accuracy and robustness of IP camera detection systems

by leveraging the strengths of each approach. However, gathering large-scale labeled datasets requires considerable time and effort, which is crucial for improving the quality of the learning model. Furthermore, effective training in ML requires sufficient computing resources and domain expertise. Additionally, when applied to a changed detection environment, the performance of the pre-trained model may degrade.

Drone detection typically involves various methods such as heat-based detection using infrared cameras, physical object detection using radar, visibility detection using optical cameras, detection of acoustic signals using sound receivers, and detection of RF signals using RF receivers [9]. However, considering that video surveillance devices like IP cameras and drones primarily use wireless communication, we aim to explore detection methods that effectively leverage this characteristic [22].

Among these methods are passive RF detection and Wi-Fi traffic pattern analysis [1], [9]. The passive RF detection method involves monitoring the electromagnetic spectrum to detect the signals exchanged between the drone and its remote control (RC) [1]. This method can be used alone or combined with ML to improve the accuracy of drone detection. Aouladhadj et al. proposed a method that captures beacon frame packets to decode the Drone ID packet, enabling remote data extraction and long-distance drone tracking [1]. However, this method requires identifying the supplier based on the UAV's MAC address, necessitating the maintenance of an up-to-date drone database. Consequently, drones that are not included in the database cannot be accurately identified, resulting in potential security risks and gaps in surveillance coverage. Another method involves capturing the RF signals of various drones and using either a CNN-based model or techniques like Fourier transforms to classify the drone signals [23]. The ML approach performs well on trained RF signals, but it may struggle with new, untrained signals. Drones typically use frequency bands that coexist with numerous other signals, such as Wi-Fi or Bluetooth. It can be challenging to classify each signal in a complex urban environment [1]. Additionally, this method requires specialized RF signal detection equipment.

Analyzing Wi-Fi traffic patterns is a promising detection method, particularly useful as most commercial drones are controlled via Wi-Fi signals. Furthermore, these drones typically offer FPV (First-Person View) video functionality, which directly streams video captured by the drone's camera to a display on a smartphone [24]. This characteristic makes Wi-Fi traffic pattern analysis a valuable approach for detecting and monitoring drones. Among the methods utilizing Wi-Fi traffic patterns, one effective approach involves using Wi-Fi probe request packets, which are continuously broadcasted by Wi-Fi devices [25], [26]. Acuna et al. extracted RSSI and MAC addresses from these packets and used this information with a Random Forest ML algorithm to determine the drone's location [25]. Another method proposed is detecting COTS (commercial

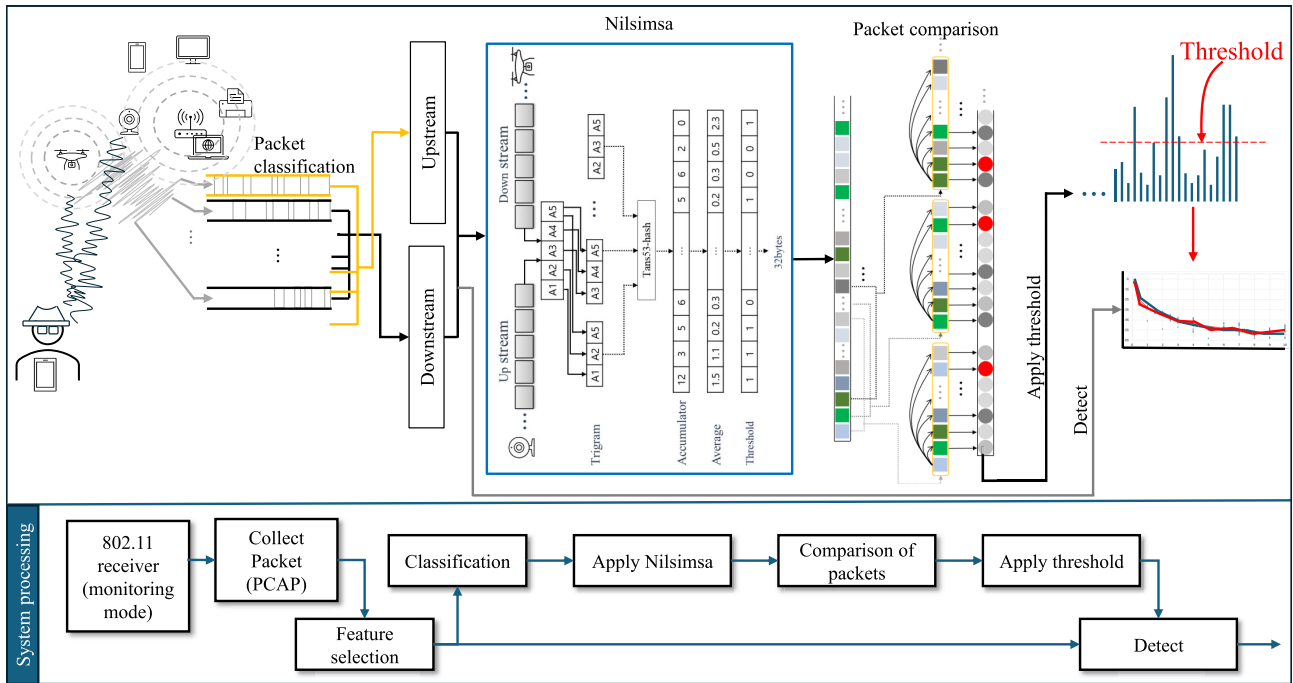


FIGURE 1. System architecture and framework.

off-the-shelf) drones by analyzing the size and inter-arrival time of packets transmitted by the drone, which are typically repetitive and of uniform length, using a Random Forest classifier trained on FPV traffic data [26]. Furthermore, drones can also be identified by analyzing unique patterns present in the video traffic they transmit, employing a Random Forest classification model for detection [27]. Nassi et al.'s methods involve blinking infrared LEDs to identify drones and determine if a point of interest (POI) is being streamed via the video channel [28].

Traffic analysis often incorporates ML techniques such as Random Forest. While Random Forest classifiers are robust, they heavily depend on extensive and diverse datasets to accurately distinguish between normal and abnormal patterns. Consequently, the need for large-scale data collection and processing can be a limitation, especially in fast-changing and dynamic environments.

Device localization techniques can be broadly categorized into three types: methods based on wireless traffic characteristics, light reflection, and signal characteristics [11].

First, methods based on wireless traffic characteristics analyze video frame size patterns, which vary depending on the distance between the spy camera and the user, to estimate the location of the device [17]. However, these methods are vulnerable to noise interference and data latency, requiring additional mitigation steps during the data processing stage. Furthermore, as they rely on detecting changes in network traffic caused by human movements, specific motions may be required to initiate the detection process [16].

Second, light reflection-based methods utilize the reflection of laser light from hidden cameras to determine the device's location. By using a smartphone's Time-of-Flight sensors, these methods detect reflected light intensities higher than the surrounding environment to pinpoint the device. However, this technique requires the user to be at a specific distance and angle, making detection challenging [11].

Third, methods based on signal characteristics include time-based techniques and signal strength indicators such as RSSI and CSI. Time-based techniques like Time of Arrival (ToA), Time Difference of Arrival (TDoA), and RTT can be used for localization. However, ToA and TDoA require strict synchronization between the transmitter and receiver, posing practical challenges. In contrast, RTT measures the time difference to estimate distance and can capture the complex relationship between human movement and time differences using deep learning [12]. However, it requires specialized transmitting and receiving equipment, which limits its practicality for general indoor localization.

Similarly, CSI provides a more accurate representation of signal strength in complex indoor environments compared to RSSI. However, it also requires specialized equipment, making them less suitable for widespread indoor use. Additionally, CSI metrics rely on detecting changes in network traffic caused by human movement, meaning that specific motions or actions are often required to effectively initiate the detection process.

Finally, RSSI is one of the most commonly used methods for indoor localization because it is easily accessible on most smart devices [13]. Although signal propagation can be

affected by phenomena such as reflection, diffraction, and shadowing, RSSI values in proximity tend to vary within acceptable ranges and closely resemble actual values, making it a useful metric for approximate localization [29].

Therefore, we aim to propose a practical solution for detecting wireless video devices and estimating their locations in real-time. Our method identifies these devices based on traffic patterns using off-the-shelf products while leveraging the RSSI values in the transmitted packets for location estimation, enabling effective real-time detection.

III. PROPOSED METHOD

This section outlines the method for detecting and identifying surveillance devices that stream video wirelessly in the vicinity of the user. Fig. 1 illustrates the proposed system architecture and framework. Each stage involves Wi-Fi sniffing for packet capture, feature selection for classification, packet grouping, application of the Nilsimsa algorithm for similarity comparison, and location tracking through the application of the LDPL model using antenna signal RSSI values. Algorithm 1 presents the pseudocode for the proposed sequence.

A. WI-FI SNIFFING SYSTEM

IEEE 802.11 specifies the set of medium access control (MAC) and physical layer (PHY) protocols for implementing wireless local area network (WLAN) computer communication [30]. The MAC layer of the 802.11 protocol operates through the exchange of request and response messages [31]. Mobile devices can access networks through two search modes: passive scanning and active probing. Access points (APs) broadcast beacons in passive mode to advertise their services to nearby clients. Mobile clients listen passively on each channel, waiting for periodic beacons from nearby APs. In active probing mode, mobile devices consistently transmit probe requests in search of previously connected networks, enabling automatic reconnection [30], [32]. In a wireless network, association means that a wireless device registers with an AP to access the network. Through this connection, data can be exchanged between the AP and the device.

Wireless sniffing systems utilize external wireless antennas equipped with monitor mode to detect nearby networks and capture Wi-Fi packets exchanged between AP and devices [30]. These two devices exchange a significant amount of data through Wi-Fi packets, which are delivered to the user space regardless of the target MAC address. By analyzing these packets, we can effectively examine the communication between the devices.

B. FEATURE SELECTION

802.11 frames or MAC Protocol Data Units (MPDUs) are structured as depicted in Fig. 2. The key fields in the MAC header are the Address fields and Frame Control. IEEE MAC standards define client and AP MAC addresses, uniquely identifying devices at the data link layer. MAC addresses,

Algorithm 1 Malicious Devices Detection

Input: Packets $Ps = (0, \dots, N)$, Threshold value **ThValue**, Packet comparison number **CompNum**, Threshold over count **ThOverCnt**

Output: Distance from detected device

```

1:  $cnt \leftarrow 0$ 
2:  $result \leftarrow False$ 
3:  $dist \leftarrow -1$ 
4: for  $i = 0$  To  $N$  do
5:    $feature[i] \leftarrow getFeature(Ps(i))$ 
6:    $Pg \leftarrow classificationPacket(Ps(i), feature[i])$ 
7:    $result, dist \leftarrow detectDevice(Pg, feature)$ 
8:   if  $result \ \& \ dist > 0$  then
9:     return  $dist$ 
10:  end if
11: end for
12: return  $dist$ 

13: function  $detectDevice(Pg, feature)$ 
14:    $detResult \leftarrow False$ 
15:   for  $i = 0$  To  $len(Pg)$  do
16:      $base \leftarrow ApplyNilsimsa(Pg(i))$ 
17:     for  $j = i + 1$  To  $CompNum$  do
18:        $comp \leftarrow ApplyNilsimsa(Pg(j))$ 
19:       if  $(j \bmod compNum) == 0$  then
20:          $cnt = 0$ 
21:         break
22:       else
23:          $hRet \leftarrow HammingD(base, comp)$ 
24:         if  $hRet > ThValue$  then
25:            $cnt + = 1$ 
26:         end if
27:       end if
28:       if  $cnt > ThOverCnt$  then
29:          $detResult = True$ 
30:          $distance = getDistance(feature)$ 
31:         break
32:       end if
33:     end for
34:     if  $detResult$  then
35:       break
36:     end if
37:   end for
38:   return  $detResult, distance$ 
39: end function

```

globally unique to each network interface card (NIC), are used for device fingerprinting and tracking. Since transmitted frames periodically expose MAC addresses, detection is feasible even when the device is not connected to an AP [33]. Address 1 contains the client's MAC address, Address 2 is the Basic Service Set Identifier (BSSID) address, Address 3 contains the AP's MAC address, and Address 4 is the source address in Fig. 2 [34].

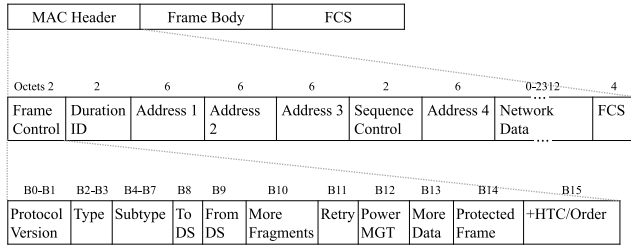


FIGURE 2. MAC header format.

The ToDS and FromDS fields in the Frame Control field indicate frame direction between the AP and client [34], [35]. It is indicated that the frame travels to/from AP if ToDS (FromDS) is set to 1. The addresses used in the MAC header are determined based on the ToDS and FromDS bits defined in the control field [36]: Address 1 is the AP's address if ToDS is set; otherwise, it's the address of the end station. Address 2 is the AP's address if FromDS is set; otherwise, it's the address of the end station. If FromDS is set to 1, Address 3 is the source address; if ToDS is set, it's the original destination address. Address 4 is used when the frame is transmitted from one AP to another in a wireless distribution system, with both ToDS and FromDS bits set. In this case, Address 4 serves as the source address.

The Retry field in the frame control field of the MAC header also plays a crucial role in detection. Initially, the Retry bit is set to 0 for all transmitted frames. If the frame is lost, the Retry bit is set to 1, indicating a retransmission with the frame. This plays an important role in our system, which verifies the similarity between packets because the similarity appears high for frames with the Retry bit set to 1 [34].

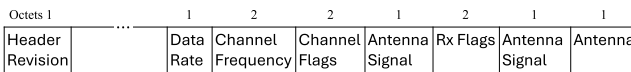


FIGURE 3. Radiotap header format.

All transmitted packets include additional information through a Radiotap header, as illustrated in Fig. 3. Among these, the antenna and antenna signal fields are utilized for tasks such as signal strength mapping, link quality estimation, and interference detection in wireless communication systems. The antenna field in the Radiotap header specifies the index of the antenna that received or transmitted the frame. The antenna signal field, measured in dBm units, is utilized for path loss-based location estimation and provides information about the strength or quality of the received signal. It typically represents the RSSI or Signal-to-Noise Ratio (SNR), indicating how well the antenna received the signal.

Thus, our system collects the address field information from the MAC header, the ToDS and FromDS bits from the frame control field, the retry field information, and the antenna field information from the Radiotap header. This information is extracted using the *getFeature* function in

line 5 of Algorithm 1, and the extracted values are stored in a *feature* array. These values are utilized for packet grouping, classification, and location estimation.

C. PACKET CLASSIFICATION

The streaming characteristics vary depending on the type of video detection device. IP cameras typically upstream the captured video footage over the Internet [3]. On the other hand, drones generate a private Wi-Fi network accessible to users. Through this Wi-Fi connection, users can control the drone using dedicated RC controllers, smartphones, or tablets, or receive the video feed captured by the drone [1]. Therefore, distinguishing between Upstream and Downstream also allows for device differentiation based on their streaming characteristics. Packet classification is performed based on the Address fields, ToDS, and FromDS fields within the Frame Control field of the MAC Header. Once collected, packets are accumulated based on pairs of Source and Destination addresses. The packets are then classified as upstream and downstream based on the values of the ToDS and FromDS fields and grouped as defined in line 6 of Algorithm 1. The packets classified in this way are then subjected to the Nilsimsa algorithm for similarity checks.

D. NILSIMSA ALGORITHM

We aim to identify packet flows that are persistently similar within the packet stream. Video typically comprises a sequence of such images, usually captured at a rate of five to 120 times per second, commonly referred to as frames per second (FPS) [37]. Additionally, each frame is not transmitted within a single packet but is divided into fragments, each transmitted separately with a maximum size of 1500 bytes, the MTU limit in wireless networks [3]. Within each frame, adjacent pixels have strong correlations [38]. Therefore, it is plausible for similarities to exist between fragmented frames. Most hash algorithms generate completely different hash values, even for slight changes in the input. In contrast, the Nilsimsa digest method reduces sensitivity to create similar hash results for similar inputs [39]. Therefore, the Nilsimsa digest method aligns well with our objective of identifying flows of packets with high similarity.

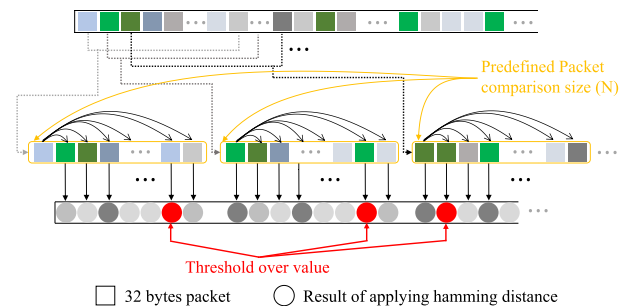


FIGURE 4. Packet comparison.

As shown in Fig. 1, packets collected from the wireless network are sequentially processed by the Nilsimsa algorithm

without any preprocessing. Five window sizes are created by reading the collected packets one byte at a time. Trigrams are then created using these windows and sent into the **Tran53 hash** function. The output of the **Tran53 hash** function produces values ranging from 0 to 255. For each generated value, the corresponding position in a 256-size accumulator is incremented by one. This process allows for storing the frequency of trigrams that pass through the **Tran53 hash** for the input packet. Subsequently, the frequencies stored in the 256-size accumulator are averaged to calculate the ratio for each frequency. Then, the median of the ratios is calculated. This part is related to the Threshold in the Nilsimsa algorithm depicted in Fig. 1. If a ratio exceeds this median, it is set to 1; otherwise, it is set to 0, resulting in the generation of a 32-byte hash. In Algorithm 1, these values are determined using the *ApplyNilsimsa* function in line 16. Packet comparison is conducted based on the 32-byte hash generated.



FIGURE 5. The test bed environment.

E. PAKET COMPARISON

In this section, we sequentially compare the 32-byte packets generated by the Nilsimsa algorithm, as shown in Fig. 4. This process is conducted in the *detectDevice* function of Algorithm 1. The comparison is conducted for a predefined packet comparison size (N). For example, comparisons are made from the 1st to the N^{th} packet, then from the 2nd packet to the $(N + 1)^{\text{th}}$ packet, and so on, iteratively comparing consecutive packets. The packets are examined using the **Hamming distance** to assess their similarity. This method counts the number of matching bits, identifying if identical values match at the same position. Finally, the similarity score is determined by subtracting 128 from the matching bit count. If the similarity scores of two packets are the same, the score is 128; if they are completely dissimilar, the score is -128 . These results are sequentially stored and transmitted as a step toward detecting malicious surveillance devices.

The transmitted results are compared against a predefined threshold to detect devices that exceed this threshold in proximity to the user. If the device frequently exceeds this threshold, it may indicate that similar packets are being transmitted periodically in the vicinity.

F. LOCALIZATION

In this section, we estimate the approximate location of a detected device when it periodically transmits similar packets. This estimation is based on the RSSI values captured from the antenna field of RadioTap packets received over the wireless network, as detailed in Section III-B, “Feature Selection.” The antenna field is included in packets transmitted over the wireless network, providing the potential for device location determination. Although RSSI values exhibit variability, they remain within an acceptable range, making them useful for estimating approximate locations.

The LDPL model, which is commonly used to define the logarithmic relationship between path loss and the distance between the transmitter and receiver, considers shadowing effects caused by objects in dense environments, such as buildings [29], [40]. The general formula is as follows.

$$PL_{d_l} = PL_{d_0} + 10\gamma \log \frac{d_l}{d_0} \quad (1)$$

In Eq. 1, d_l represents the path length from the transmitter to location l , while d_0 denotes the reference distance. The PL_{d_l} signifies the path loss at distance d_l , and PL_{d_0} corresponds to the signal strength at the reference distance d_0 . γ is the path loss exponent. The distance d can be calculated using Eq. 1 as follows.

$$d = d_0 \times 10^{\frac{PL_{d_l} - PL_{d_0}}{10\gamma}} \quad (2)$$

TABLE 2. Device specifications.

Device	Category	Specification
Camera 1	Model name	Starcam 100a
	Digital Format	Multistream 720P / VGA / QVGA 30FPS
	Power consumption	2.5 W
	Horizontal view angle	270°
	Vertical view angle	80°
Camera 2	Model name	Finevu k50
	Digital Format	Full HD
	Power consumption	5.4 W
	Horizontal view angle	355°
	Vertical view angle	52°
Drone 1	Model name	Mini 3 z908 pro
	Remote control frequency	2.4GHz / 4 channels
	Flight time	about 15 min
	Fuselage battery	3.7V 2000 mAh
	Remote control distance	200 m
	Body size	13 × 9 × 5
	Image transmission signal	Wi-Fi signal
	UAV weight(including battery)	138 g
	Adjust the camera	148°
Smartphone	Model name	Samsung Galaxy S21
	Android version	14

Based on Eq. 2, we can estimate the distance d .

IV. EXPERIMENT ENVIRONMENT

This section outlines our experimental setup for identifying video transmission devices.

A. EXPERIMENTAL SETUP

As shown in Fig. 5, the setup was tested with two stationary IP cameras and a single mobile camera (drone). Additionally, the components used for detection are as follows: The access point (AP) is a low-cost, entry-level, off-the-shelf device designed for general home users. To monitor the Wi-Fi traffic, a Raspberry Pi 4 Model B and an ALFA AWUS036NHA USB 2.0 adapter are utilized. These devices are labeled as 5 and 4 in Fig. 5, respectively. While this model captures all packets exchanged between the AP and clients, its primary limitation is its outdated module, which restricts it to the 2.4GHz band. Therefore, we will focus exclusively on the 2.4GHz band. The Raspberry Pi is installed with Kali Linux, a Debian-based operating system that provides a suite of open-source tools designed for advanced penetration testing, security scanning, and information security-related tasks. Various tools facilitate security-related tasks such as penetration testing, vulnerability management, and security research. Network traffic is collected using a wireless network adapter with monitor mode enabled and the Aircrack-ng toolbar available in Kali Linux. Three mobile phones connected to each device are used to receive the video stream. The first camera, 100a, labeled as number 1 in Fig. 5, is connected to the smartphone depicted as number 6. The second camera, K50, labeled as number 2, is connected to the smartphone depicted as number 7. The drone is connected to the smartphone labeled as number 8 via their respective applications. The specifications for each device are detailed in Table 2. Furthermore, experiments in a hallway 2.28 meters wide and 10 meters tall are conducted to measure RSSI by distance.

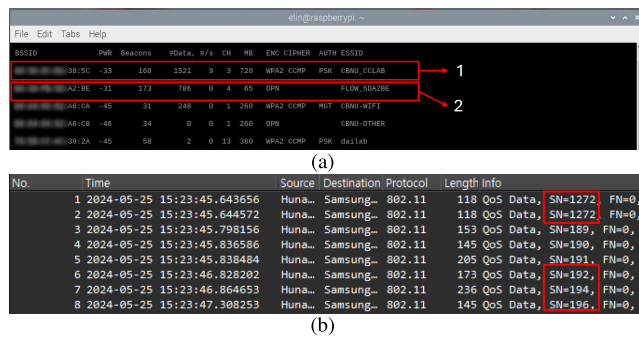


FIGURE 6. Packet capture results: (a) Comparison of packet sizes and (b) Capture times and sequence numbers of packets.

B. SETUP FOR PACKET CAPTURE

In the environment configured as shown in Fig. 5, the versatile aircrack-ng and wireshark tools are employed to monitor nearby Wi-Fi networks. The aircrack-ng enables the reading and scanning of MAC addresses (BSSIDs) of wireless devices

Length	Info	Length	Info	Length	Info
1182	1234 → 38364 Len=1080	768	1234 → 38364 Len=666	619	1234 → 38364 Len=517
1093	1234 → 38364 Len=991	876	1234 → 38364 Len=774	674	1234 → 38364 Len=572
1085	1234 → 38364 Len=983	889	1234 → 38364 Len=787	920	1234 → 38364 Len=818
1065	1234 → 38364 Len=963	832	1234 → 38364 Len=730	707	1234 → 38364 Len=605
1050	1234 → 38364 Len=948	666	1234 → 38364 Len=564	647	1234 → 38364 Len=545
894	1234 → 38364 Len=792	661	1234 → 38364 Len=559	710	1234 → 38364 Len=608
966	1234 → 38364 Len=864	807	1234 → 38364 Len=705	898	1234 → 38364 Len=796
1060	1234 → 38364 Len=958	800	1234 → 38364 Len=698	776	1234 → 38364 Len=674
879	1234 → 38364 Len=777	865	1234 → 38364 Len=763	757	1234 → 38364 Len=655
1066	1234 → 38364 Len=964	908	1234 → 38364 Len=806	719	1234 → 38364 Len=617
869	1234 → 38364 Len=767	801	1234 → 38364 Len=699	842	1234 → 38364 Len=740

FIGURE 7. Results of captured packets of various sizes.

near the user, as well as the sniffing of beacons and data packets transmitted by these devices. Wireshark is utilized to inspect packets transmitted across networks. Upon running Wireshark, it captures not only UDP packets used for sending video from the drone to the controller and data packets from IP cameras but also all packets within the network.

V. PERFORMANCE EVALUATION

In this section, we present the experimental results obtained by applying our method to detect video surveillance devices.

A. COLLECT PACKETS AND APPLY NILSIMSA

We conducted our packet collection in a real-world environment without restricting the conditions. The results of approximately 1 minute of sniffing are depicted in Fig. 6(a). Due to the two IP cameras utilizing the AP for upstream transmission, numerous packets are captured, as indicated in Fig. 6(a), number 1. Additionally, this AP, functioning as a central hub for data exchange among various devices, results in a substantial amount of captured data. Furthermore, Fig. 6(a), number 2, illustrates packets exchanging data between a drone and a smartphone. The first IP camera transmits video images in VGA format (640×360 pixels, a color depth of 3 (RGB)). Given that the Ethernet Maximum Transmission Unit (MTU) is 1500 bytes, the image data is divided into multiple packets for transmission. The second IP camera and the drone transmit images in FULL HD format. This image data also exceeds the MTU size and is therefore divided into multiple packets for transmission. As a result, both the cameras and the drone generate the largest data volume among the nearby traffic.

All wireless data is encrypted during transmission, requiring special techniques to inspect its contents [27]. However, our approach involves grouping collected data by source and destination MAC addresses and then applying the Nilsimsa algorithm without the need for special preprocessing or content inspection. This method allows us to effectively analyze the transmitted data without the complexities typically associated with preprocessing or content inspection. For example, Fig. 7 illustrates packets of different sizes from a total of 1,111 data packets transmitted from the drone to the smartphone over approximately one minute, excluding control packets. Most of the packet sizes are 1080, but about 3% of the packets are of different sizes. We directly apply the Nilsimsa algorithm to each collected

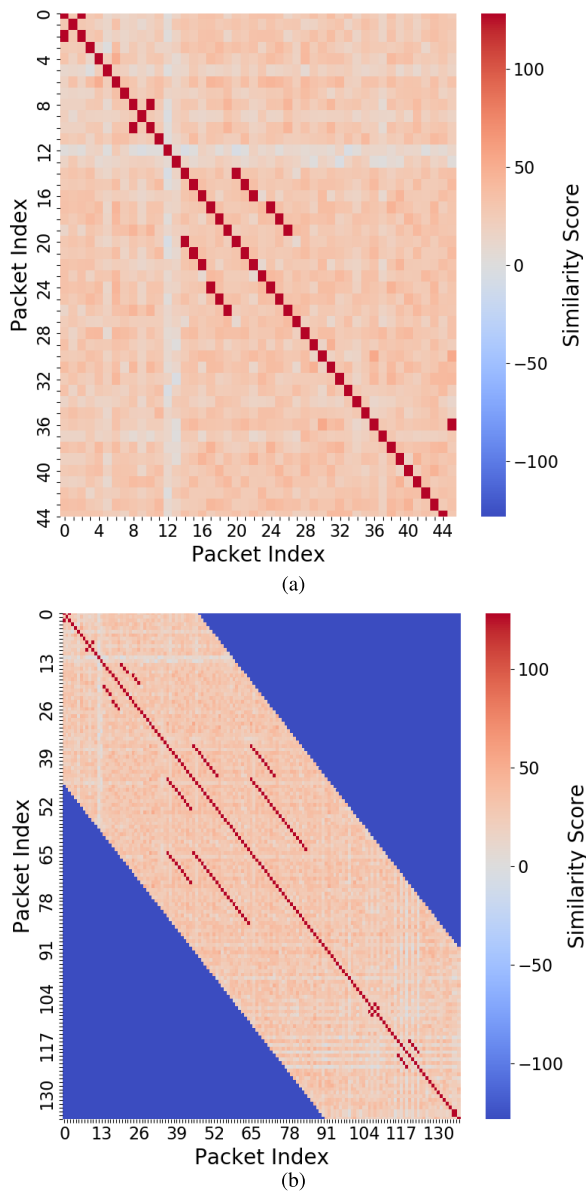


FIGURE 8. Heatmap of similarity for camera 1 in no action state: (a) Comparison of first 46 Packets and (b) Comparison of 138 Packets.

and grouped packet instead of normalizing these packet sizes. Additionally, as shown in Fig. 6(b), we do not perform any specific tasks for duplicate packets or packet loss. All packets are processed in the order they are collected, with the Nilsimsa algorithm applied to each packet. As a result, these packets are transformed into 32-byte digests. Subsequently, packet comparison is performed within each group.

B. PACKET ANALYSIS

Fig. 8 explicitly illustrates the similarity of packets by employing heatmaps, prior to conducting comprehensive packet collection and comparison. Fig. 8(a) represents the heatmap illustrating the similarity between the first collected packet and the subsequent 46 packets for Camera 1 when

no action occurs. The red areas along the diagonal represent comparisons of packets with the same index, indicated by a value of 128. The lower-left and upper-right sections also show identical values, as they compare packets with matching indices, which is a characteristic of heatmap visualization. This result indicates that the 46 packets predominantly exhibit similarity scores exceeding 30, with 9 packets showing complete similarity. Except for the diagonal red points, the red points represent similarity scores close to 128.

Fig. 8(b) compares a total of 138 packets, with each packet being compared to a maximum of 46 others. This corresponds to the comparison of three groups, each enclosed in an orange box in Fig. 4, representing the predefined packet comparison size (46). Therefore, the blue areas in the lower-left and upper-right sections signify that no similarity comparisons were conducted, resulting in a value of -128 , thus represented in blue. As demonstrated in this result, packet similarity can be observed even with a limited number of packet comparisons.

We present the results of comparing approximately 1,100 sequentially collected packets in Fig. 9. These packets were captured within a timeframe of less than 3 seconds. We conducted tests under three conditions: a stationary scene (no action), less than 20% motion (small action) on the screen, and more than 90% motion (big action) across the entire screen. Fig. 9 (a), (b), and (c) show the results for Camera 1, a Starcam 100a model, while Fig. 9 (d), (e), and (f) show the results for Camera 2, a Finevu k50 model. These results demonstrate that even though both are IP cameras, the packet format, transmission speed, and vendor can cause different outcomes. Additionally, the results for a drone tested under the same three conditions are shown in Fig. 9 (g), (h), and (i).

Examining the packet similarity results presented in Fig. 9 reveals that some packets exhibit a similarity score of 128, indicating that these two packets are identical and are observed repeatedly. This finding correlates with the complete similarity indicated by the red pixels, which represent identical packets in Fig. 8(a) and 8(b). In cases where the similarity score is lower, most packets tend to have scores below 50.

The results shown in Fig. 9 are obtained from packet comparison using a fixed packet comparison size of 46. We conducted additional experiments with packet comparison sizes of 20 and 100 to investigate whether the packet comparison size influences the rare occurrences of packet similarity seen in Fig. 9 (d), (e), and (f). The results of these experiments are shown in Fig. 10. The results indicate that smaller packet sizes tend to show similarities more frequently. Specifically, the analysis of one packet takes 2.057×10^{-5} seconds, and comparing 100 packets takes approximately 0.002 seconds. Our goal is to complete the comparison analysis before the arrival of the next packet and to maximize the number of packets compared. As shown in Fig. 6(b), the packets arrive in less than 0.001 seconds. Therefore, it is preferable to limit the comparison to packets

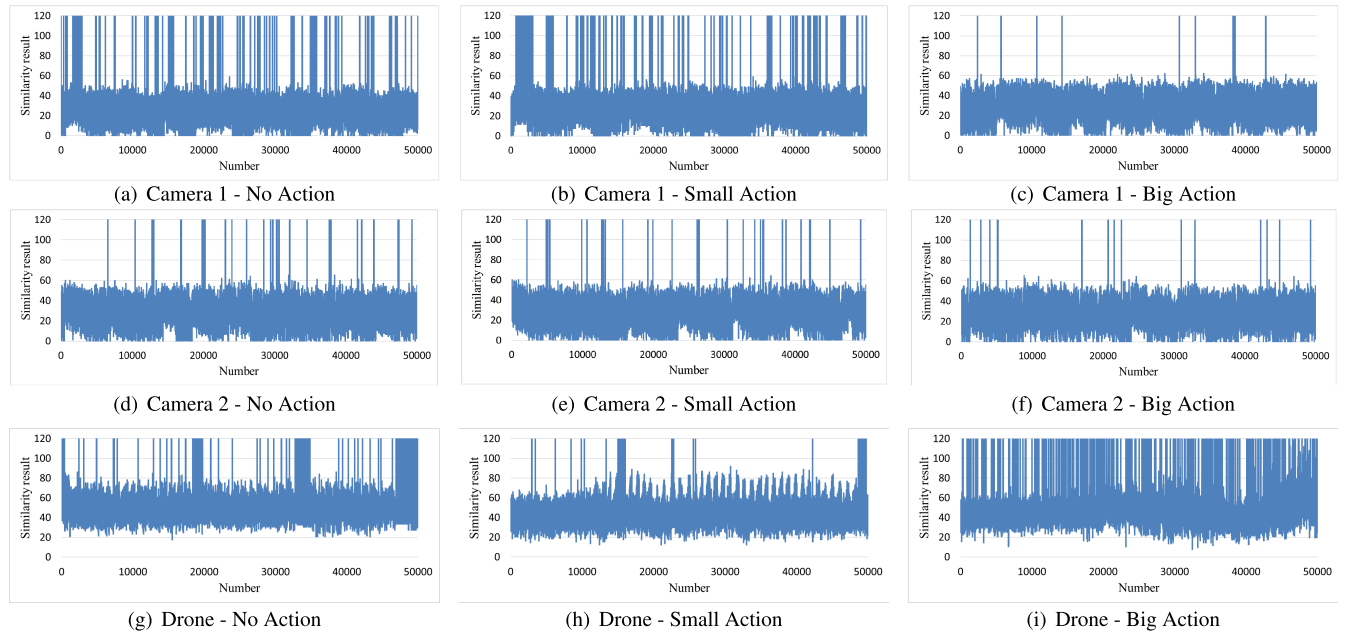


FIGURE 9. The comparison results according to movement: (a) Camera 1 - No action, (b) Camera 1 - Small action, (c) Camera 1 - Big action, (d) Camera 2 - No action, (e) Camera 2 - Small action, (f) Camera 2 - Big action, (g) Drone - No action, (h) Drone - Small action, (i) Drone - Big action.

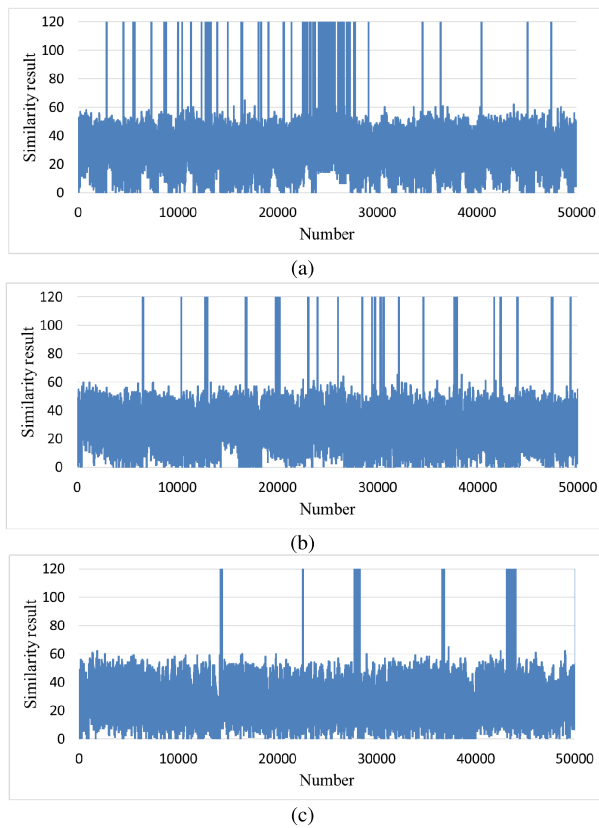


FIGURE 10. The results according to data comparison size P_s from camera 2: (a) Comparison size (20), (b) Comparison size (46), and (c) Comparison size (100).

of 50 or fewer. However, if the packet size is too small, the comparison may only cover specific parts of the image. This

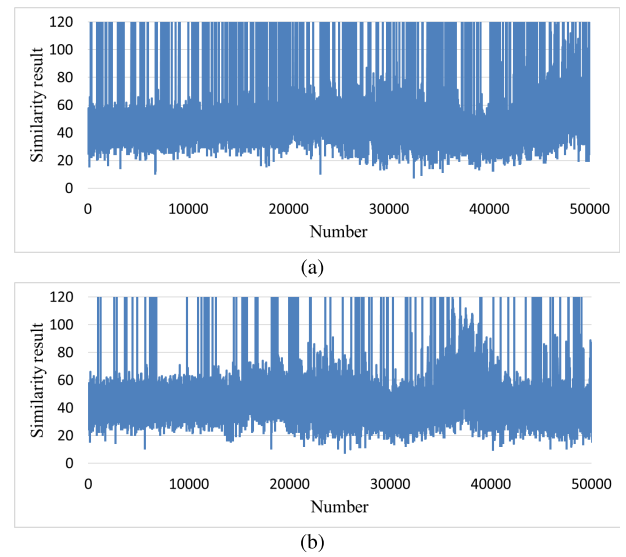


FIGURE 11. Results of drones before and after removing duplicate packets: (a) Before (b) After.

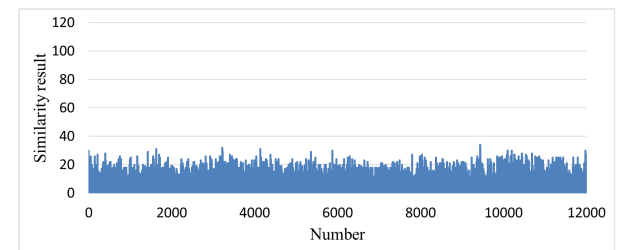


FIGURE 12. Comparison result of different device.

leads to the conclusion that a packet comparison size of 46 is appropriate.

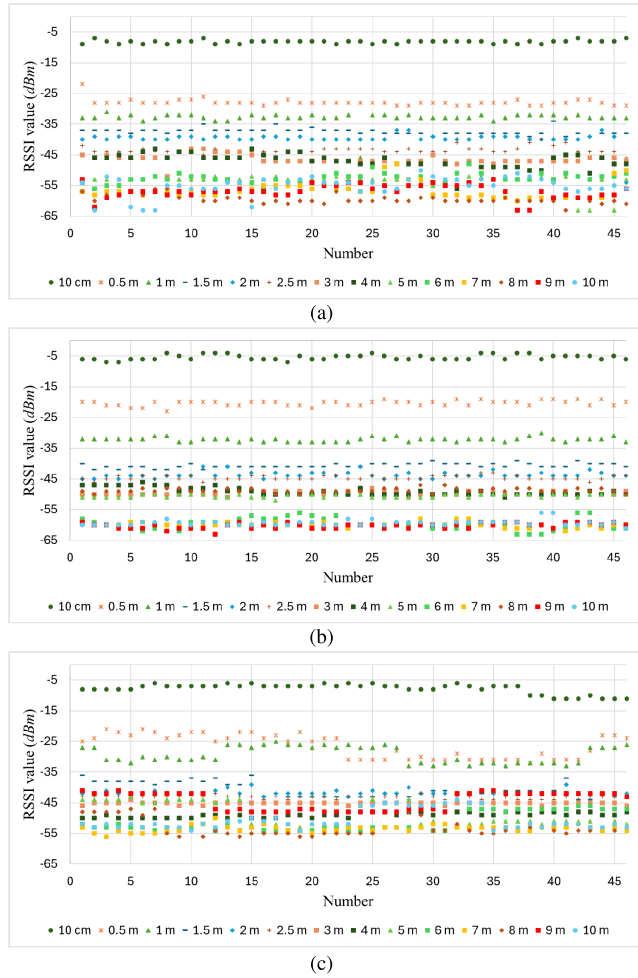


FIGURE 13. Distance-based Wi-Fi RSSI values collected from each device: (a) Camera 1 (Startcam), (b) Camera 2 (Finevu), (c) Drone (Mini 3).

We also investigated the impact of duplicate packets. Wireless channels can cause irregular video transmission and packet loss due to interference from other wireless devices and general channel noise, as shown in Fig. 6(b) [41]. Consequently, duplicate packets can vary with the quality of the wireless network. We examined whether packet similarity occurs even in the absence of duplicate packets. Specifically, we used drone footage that, despite significant movement, maintained high similarity to analyze the impact of duplicate packets on this similarity, as shown in Fig. 9(i). To eliminate duplicate packets, we checked the Retry bit in the Frame Control field of the MAC header, as illustrated in Fig. 2. If this bit was set to 1, indicating the packet had been retransmitted, we discarded the packet. The results are shown in Fig. 11. Out of the 15,775 packets collected, 3,616 were duplicates, accounting for 22.9% of the packets. In contrast, during periods of no movement, as depicted in Fig. 9(g), out of 17,029 packets, only 1,211 were duplicates, accounting for approximately 7% duplication. Moreover, for IP cameras, which are typically installed in fixed positions, less than 2.7% duplication (42 duplicate packets out of 1,701 packets)

occurred during packet capture in periods of significant movement. The stability of fixed IP cameras results in fewer retransmissions and consequently lower duplication rates. This finding suggests that higher packet duplication rates are more prominent in scenarios involving substantial movement, such as with drones, likely due to increased packet loss.

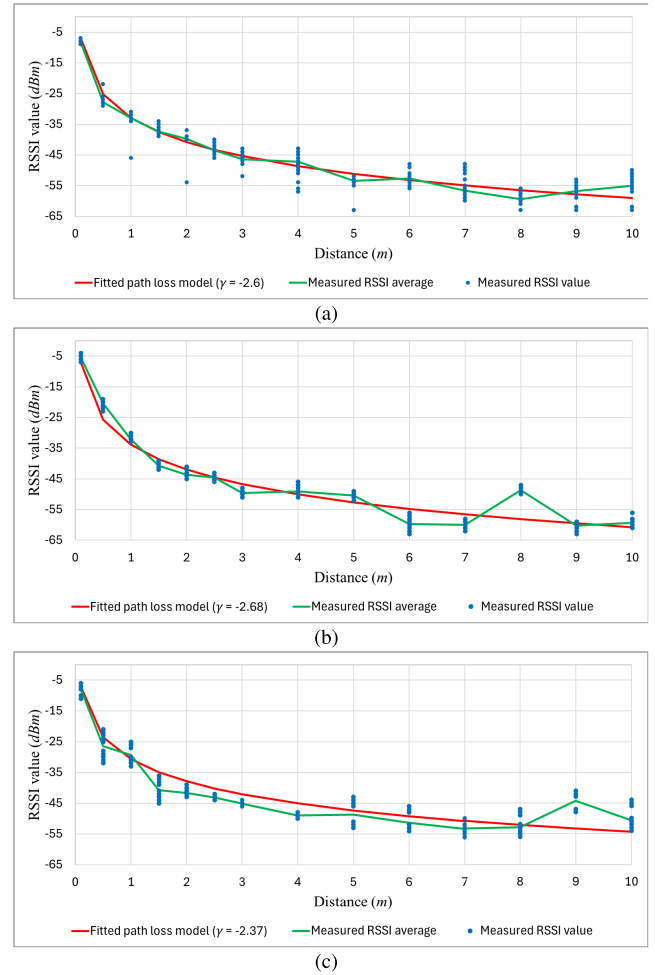


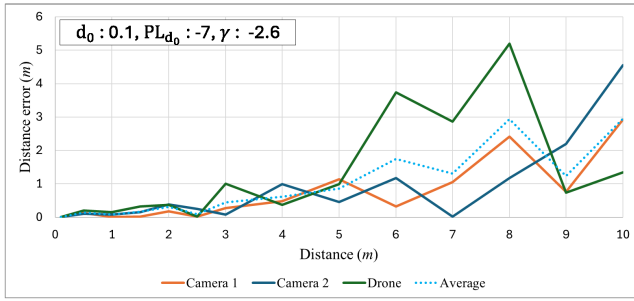
FIGURE 14. RSSI distribution and mean measurements, fitted LDPL model results by distance: (a) Camera 1 (Startcam), (b) Camera 2 (Finevu), (c) Drone (Mini 3).

Furthermore, we conducted additional experiments to explore how similarity results were generated across different devices. We selected packets exchanged between devices other than the two cameras and the drone, with the collected packets originating from the device with the fourth-highest packet exchange volume. The results obtained by applying our algorithm to the collected packets can be observed in Fig. 12. While other data involved packet captures within one minute, this specific result required over 10 minutes of data collection to accumulate a total of 12,000 packets. As illustrated in the figure, no data similarity was observed among the total of 12,000 packets. The differences become strikingly clear when comparing the similarity scores with those of the video transmission devices in Fig. 9. By applying

TABLE 3. Comparison of RSSI average value comparison by location (dBm) and distance error after applying LDPL (m).

	Device	0.1 m	0.5 m	1 m	1.5 m	2 m	2.5 m	3 m	4 m	5 m	6 m	7 m	8 m	9 m	10 m
RSSI	Camera 1	-8.17	27.87	-32.96	-37.41	-39.74	-43.46	-46.39	-47.2	-53.5	-52.6	-56.57	-59.46	-56.8	-55.1
	Camera 2	-5.39	-20.35	-32.15	-40.65	-43.7	-44.63	-49.6	-49.04	-50.46	-59.65	-59.98	-48.74	-60.24	-59.3
	Drone	-7.76	-26.52	29.3	-40.8	41.7	-43.2	-45.13	-49	-48.67	-51.35	53.4	-52.9	-44.39	-50.65
LDPL	Camera 1	0.01	0.13	0	0.02	0.18	0.02	0.27	0.48	1.14	0.32	1.06	2.41	0.77	2.93
	Camera 2	0.01	0.11	0.08	0.16	0.39	0.25	0.08	1	0.46	1.18	0.02	1.18	2.19	4.55
	Drone	0.01	0.2	0.15	0.33	0.37	0.02	1	0.37	0.99	3.74	2.86	5.2	0.73	1.35
	Average	0.01	0.15	0.08	0.17	0.32	0.1	0.45	0.62	0.87	1.75	1.31	2.93	1.23	2.94

the threshold of 70, as suggested in previous studies, it is deemed appropriate for detecting malicious devices, irrespective of the movement status of the three devices depicted in Fig. 9 [3]. Consequently, this study also employs this threshold to detect surveillance video devices.

**FIGURE 15.** Measured distance error for each device using LDPL method.

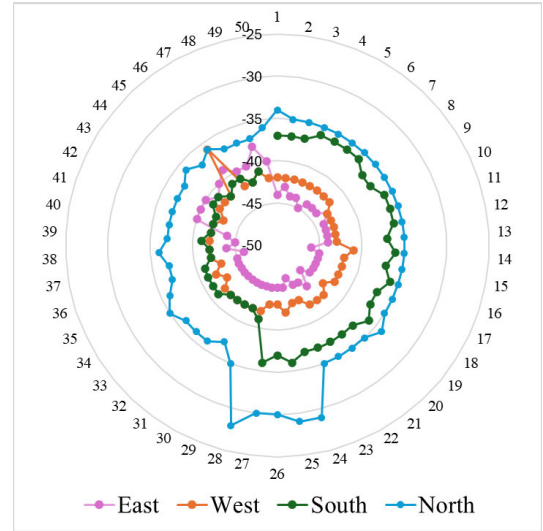
C. COLLECT AND ANALYSIS RSSI VALUES

This session is dedicated to determining the approximate position of any identified video streaming devices that are near the user. As explained in Fig. 1, as packets are captured, their required features are selected, and based on these features, the data's RSSI values are obtained. To convert the antenna signal value in the Radiotap header (shown in Fig. 3) into the actual RSSI value we use, we subtract 256 from the read-byte value. Any outliers in the collected RSSI values, exceeding -100 or more than 100% greater than the median of the collected RSSI, are removed.

One significant advantage of our algorithm is that while conducting similarity checks, we can simultaneously match localization, allowing us to approximate the position of the device while analyzing packets.

We collected the RSSI values of each device at 14 positions, ranging from 0.1 m to 10 m . The results can be seen in Fig. 13, and the average values for each distance are available in Table 3. These RSSI values are obtained from packets captured in a no-action scenario. Although there is some fluctuation in the RSSI values, it is generally

observed that the differences in RSSI values at different distances are evident for all devices. However, as the distance increases, the difference in RSSI values becomes less pronounced. Additionally, at distances beyond 7 m , an anomalous phenomenon occurs where the RSSI values, instead of decreasing, increase. In these instances, the percentage of outlier RSSI values exceeds 50% .

**FIGURE 16.** Comparison of RSSI values in four directions at a distance of 1.5 m based on the antenna direction of the USB Wi-Fi adapter.

Based on the collected results, we applied the LDPL model and observed the outcomes shown in Fig. 14. This Fig. 14 illustrates the distribution of collected RSSI values at various distances and the results after applying the LDPL model. Applying Eq. 1 to these three devices yielded path loss exponent (γ) results of -2.6 for Camera 1, -2.68 for Camera 2, and -2.37 for the drone. Using these values, we applied Eq. 2 to measure the distance to the device based on the average of the collected RSSI values. The initial reference distance (d_0) was set to 0.1 m , and the signal strength at the reference distance (PL_{d_0}) used was -7 , which is the average of the initial RSSI values for the three devices. Additionally,

we used a path loss exponent (γ) value of -2.6, which is close to the average of the three path loss exponent values. The absolute measured distance is then compared to the actual distance. The results are presented in Fig. 15 and Table 3. The results are presented in Fig. 15 and Table 3. They have a distance error of less than 0.39 m within a measured distance of 2.5 m. The results show an average distance error of about 0.08 m within a 1 m range and about 0.45 m within a 3 m range. However, once the distance exceeds 5 m, a significant distance error is observed between the actual position of the drone and the measured distance. Nonetheless, within a distance of 10 m, the average distance error remains within 3 m. This is primarily due to the difference between the path loss exponent we set and the drone's actual path loss exponent value.

Additionally, we conducted tests to investigate the changes in RSSI values based on the antenna orientation of the Wi-Fi USB adapter. This test was performed using a drone. The antenna, oriented towards the East, and the drone, positioned in the same direction, were used to measure RSSI values from each of the four cardinal directions at a distance of 1.5 m. The results are shown in Fig. 16. The findings indicate that the RSSI values were highest when measured from the "North" direction, while surprisingly, the RSSI values were lowest when measured from the "East" direction, the same direction the antenna was facing. This suggests that the antenna's performance is not necessarily optimal in the direction it is facing. Despite this, this information can be valuable for determining the orientation of the antenna to achieve optimal performance in localizing video devices.

VI. CONCLUSION AND FUTURE WORK

Our approach involves analyzing the network traffic surrounding the user, and by applying the Nilsimsa hashing technique, we detect the presence of devices transmitting video based on hash similarity. This method leverages various devices' unique data transmission patterns, including stationary and mobile cameras, that wirelessly transmit video data. It provides a powerful framework for effective detection without the need to decrypt the content. Additionally, while determining the presence of a device, we use the LDPL model to approximate the device's on-site location based on specific features within the packet, such as the antenna signal value. Our method can detect all wireless video devices, not just specific types. It can pinpoint devices with an average distance error of only 0.45 m within a range of up to 3 m. As a result, our approach offers a practical and efficient method for real-time detection of devices wirelessly transmitting video data, utilizing off-the-shelf equipment without the need for large amounts of data or specialized hardware.

Future studies aim to include comparative analyses of a wider variety of video devices across diverse environmental conditions. We will explore a variety of device types and transmission protocols further to validate the robustness and versatility of our detection method. Additionally, we plan to enhance our algorithms and models to improve accuracy

and extend the detection range. Furthermore, we will focus on exploring methods to enhance the precision of direction accuracy in our subsequent research.

REFERENCES

- [1] D. Aouladhadj, E. Kpre, V. Deniau, A. Kharchouf, C. Gransart, and C. Gaquière, "Drone detection and tracking using RF identification signals," *Sensors*, vol. 23, no. 17, p. 7650, Sep. 2023.
- [2] L. Cao and H. Wang, "Research on UAV network communication application based on 5G technology," in *Proc. 3rd Int. Conf. Electron. Commun. Artif. Intell. (IWECAI)*, Zhuhai, China, Jan. 2022, pp. 125–129.
- [3] H. An, W. Park, and S. Park, "Wireless spy camera spotter system with real-time traffic similarity analysis and WiFi signal tracing," *IEEE Access*, vol. 12, pp. 4459–4470, 2024.
- [4] A. Boonsongsrikul and J. Eamsaard, "Real-time human motion tracking by Tello EDU drone," *Sensors*, vol. 23, no. 2, p. 897, Jan. 2023.
- [5] M. Emimi, M. Khaleel, and A. Alkrash, "The current opportunities and challenges in drone technology," *Int. J. Electr. Eng. Sustain.*, vol. 1, no. 3, pp. 74–89, Jul. 2023.
- [6] M. Zhang and X. Li, "Drone-enabled Internet-of-Things relay for environmental monitoring in remote areas without public networks," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7648–7662, Aug. 2020.
- [7] M. Salman, N. Dao, U. Lee, and Y. Noh, "CSI: DeSpy: Enabling effortless spy camera detection via passive sensing of user activities and bitrate variations," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 6, no. 2, pp. 1–27, Jul. 2022.
- [8] I. Bisio, C. Garibotto, H. Haleem, F. Lavagetto, and A. Sciarone, "On the localization of wireless targets: A drone surveillance perspective," *IEEE Netw.*, vol. 35, no. 5, pp. 249–255, Sep. 2021.
- [9] S. Park, H. T. Kim, S. Lee, H. Joo, and H. Kim, "Survey on anti-drone systems: Components, designs, and challenges," *IEEE Access*, vol. 9, pp. 42635–42659, 2021.
- [10] S. Al-Emadi and F. Al-Senaïd, "Drone detection approach based on radio-frequency using convolutional neural network," in *Proc. IEEE Int. Conf. Informat., IoT, Enabling Technol. (ICIOT)*, Doha, Qatar, Feb. 2020, pp. 29–34.
- [11] Y. Ma, X. Luo, R. Li, S. Du, and W. Liu, "LENSER: A channel state information based indoor localization scheme for malicious devices," in *Proc. IEEE 20th Int. Conf. Mobile Ad Hoc Smart Syst. (MASS)*, vol. 45, Toronto, ON, Canada, Sep. 2023, pp. 461–470.
- [12] M. Mohsen, H. Rizk, H. Yamaguchi, and M. Youssef, "LocFree: WiFi RTT-based device-free indoor localization system," in *Proc. 2nd ACM SIGSPATIAL Int. Workshop Spatial Big Data AI Ind. Appl.*, New York, NY, USA, ACM, 2023, pp. 32–40.
- [13] J. Bi, J. Wang, H. Cao, G. Yao, Y. Wang, Z. Li, M. Sun, H. Yang, J. Zhen, and G. Zheng, "Inverse distance weight-assisted particle swarm optimized indoor localization," *Appl. Soft Comput.*, vol. 164, Oct. 2024, Art. no. 112032.
- [14] A. Fakhreddine, C. Raffelsberger, M. Sende, and C. Bettstetter, "Experiments on drone-to-drone communication with Wi-Fi, LTE-A, and 5G," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Rio de Janeiro, Brazil, Dec. 2022, pp. 904–909.
- [15] F. Li, X. Wang, Z. Wang, J. Cao, X. Liu, Y. Bi, W. Li, and Y. Wang, "A local communication system over Wi-Fi direct: Implementation and performance evaluation," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5140–5158, Jun. 2020.
- [16] R. Cunningham and W. L. Tan, "Detection and localization of hidden Wi-Fi cameras," in *Proc. 27th Asia-Pacific Conf. Commun. (APCC)*, Jeju Island, South Korea, Oct. 2022, pp. 12–17.
- [17] J. Heo, S. Gil, Y. Jung, J. Kim, D. Kim, W. Park, Y. Kim, K. G. Shin, and C.-H. Lee, "Are there wireless hidden cameras spying on me?" in *Proc. 38th Annu. Comput. Secur. Appl. Conf. (ACSAC)*, vol. 2, New York, NY, USA, Dec. 2022, pp. 714–726.
- [18] A. D. Singh, L. Garcia, J. Noor, and M. Srivastava, "I always feel like somebody's sensing me! A framework to detect, identify, and localize clandestine wireless sensors," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 1829–1846.
- [19] Y. He, Q. He, S. Fang, and Y. Liu, "MotionCompass: Pinpointing wireless camera via motion-activated traffic," in *Proc. Annu. Int. Conf. Mobile Syst., Appl., Services (MobiSys)*, New York, NY, USA, ACM, 2021, pp. 215–227.

- [20] Y. Cheng, X. Ji, T. Lu, and W. Xu, "On detecting hidden wireless cameras: A traffic pattern-based approach," *IEEE Trans. Mobile Comput.*, vol. 19, no. 4, pp. 907–921, Apr. 2020.
- [21] Z. Shi, H. Wu, J. Zhang, M. Zhang, and W. Huang, "FindSpy: A wireless camera detection system based on pre-trained transformers," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Gammarth, Tunisia, Jul. 2023, pp. 816–822.
- [22] M. A. S. Al-Nuaimi and A. A. Ibrahim, "Analyzing and detecting the de-authentication attack by creating an automated scanner using scapy," *Int. J. Recent Innov. Trends Comput. Commun.*, vol. 11, no. 2, pp. 131–137, Mar. 2023.
- [23] I. Garvanov, D. Kanev, M. Garvanova, and V. Ivanov, "Drone detection approach based on radio frequency detector," in *Proc. Int. Conf. Automatics Informat. (ICAI)*, Varna, Bulgaria, Oct. 2023, pp. 230–234.
- [24] I. Bisio, C. Garibotto, F. Lavagetto, A. Sciarone, and S. Zappatore, "Blind detection: Advanced techniques for WiFi-based drone surveillance," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 938–946, Jan. 2019.
- [25] V. Acuna, A. Kumbhar, E. Vattapparamban, F. Rajabli, and I. Guvenc, "Localization of WiFi devices using probe requests captured at unmanned aerial vehicles," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, San Francisco, CA, USA, Mar. 2017, pp. 1–6.
- [26] A. Alsoliman, G. Rigoni, M. Levorato, C. Pinotti, N. O. Tippenhauer, and M. Conti, "COTS drone detection using video streaming characteristics," in *Proc. 22nd Int. Conf. Distrib. Comput. Netw. (ICDCN)*, New York, NY, USA, Jan. 2021, pp. 166–175.
- [27] A. Alsoliman, G. Rigoni, D. Callegaro, M. Levorato, C. M. Pinotti, and M. Conti, "Intrusion detection framework for invasive FPV drones using video streaming characteristics," *ACM Trans. Cyber-Phys. Syst.*, vol. 7, no. 2, pp. 1–29, Apr. 2023.
- [28] B. Nassi, R. Ben-Netanel, A. Shamir, and Y. Elovici, "Drones' cryptanalysis—smashing cryptography with a flicker," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 1397–1414.
- [29] P. Wang and Y. Luo, "Research on WiFi indoor location algorithm based on RSSI ranging," in *Proc. 4th Int. Conf. Inf. Sci. Control Eng. (ICISCE)*, Changsha, China, Jul. 2017, pp. 1694–1698.
- [30] Y. Li, J. Barthélemy, S. Sun, P. Perez, and B. Moran, "A case study of WiFi sniffing performance evaluation," *IEEE Access*, vol. 8, pp. 129224–129235, 2020.
- [31] Al Petrick and B. O'Hara, *IEEE 802.11 Handbook: A Designer's Companion*. Piscataway, NJ, USA: IEEE Standards Association, 2005.
- [32] D. N. Ratnayake, H. B. Kazemian, S. A. Yusuf, and A. B. Abdullah, "An intelligent approach to detect probe request attacks in IEEE 802.11 networks," in *Proc. Int. Conf. Eng. Appl. Neural Netw.* Berlin, Germany: Springer, 2011, pp. 372–381.
- [33] P. Robyns, B. Bonné, P. Quax, and W. Lamotte, "Noncooperative 802.11 MAC layer fingerprinting and tracking of mobile devices," *Secur. Commun. Netw.*, vol. 2017, no. 1, pp. 1–21, 2017.
- [34] M. Agarwal, S. Biswas, and S. Nandi, "An efficient scheme to detect evil twin rogue access point attack in 802.11 Wi-Fi networks," *Int. J. Wireless Inf. Netw.*, vol. 25, no. 2, pp. 130–145, Jun. 2018.
- [35] A. Abdelrahman, H. Khaled, E. Shaaban, and W. S. Elkilani, "Detailed study of WLAN PSK cracking implementation," in *Proc. 15th Int. Conf. Comput. Eng. Syst. (ICCES)*, Cairo, Egypt, Dec. 2020, pp. 1–6.
- [36] V. Kumar, S. Chakraborty, F. A. Barbhuiya, and S. Nandi, "Detection of stealth man-in-the-middle attack in wireless LAN," in *Proc. 2nd IEEE Int. Conf. Parallel, Distrib. Grid Comput.*, Solan, India, Dec. 2012, pp. 290–295.
- [37] B. Nassi, R. Ben-Netanel, A. Shamir, and Y. Elovici, "Game of drones—detecting streamed POI from encrypted FPV channel," 2018, *arXiv:1801.03074*.
- [38] X. Chang, J. Wu, and F. Hao, "JPEG fragment carving based on pixel similarity of MED_ED," in *Proc. Chin. Control Conf. (CCC)*, vol. 23, Guangzhou, China, Jul. 2019, pp. 8862–8866.
- [39] B. Charyyev and M. H. Gunes, "Locality-sensitive IoT network traffic fingerprinting for device identification," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1272–1281, Feb. 2021.
- [40] D. P. Grabowsky, J. M. Conrad, and A. F. Browne, "Limited log-distance path loss model path loss exponent estimation using deep deterministic policy gradient," in *Proc. SoutheastCon*, Atlanta, GA, USA, 2021, pp. 1–6.
- [41] K. N. Mohammed, "FEC methods' effects on intelligent video streaming over 2.4 GHz wireless channels: An analysis," *Int. J. Intell. Syst. Appl. Eng.*, vol. 11, no. 11s, pp. 445–455, 2023.



HYEYOUNG AN received the B.S. degree from the College of Computer Science, Chungnam National University, South Korea, in 2001, and the M.S. degree from the Department of Electrical and Computer Engineering, Chungbuk National University, in 2023. She is currently pursuing the Ph.D. degree with the Department of Computer Engineering, Chungbuk National University. Her research interests include networking technologies, sensing and data analytics, optimization, AI, and the IoT.



WOOJIN PARK (Student Member, IEEE) received the B.S. degree from the School of Computer Engineering, Chungbuk National University, South Korea, in 2023. He is currently pursuing the master's degree with the Department of Computer Engineering, Chungbuk National University.



SOOCHANG PARK (Member, IEEE) received the Ph.D. degree from Chungnam National University, South Korea, in 2011. He was with the Institut Mines-Telecom, Telecom SudParis, France, as a Research Engineer, from 2013 to 2015, and Rutgers University, United States, as a Postdoctoral Researcher, in 2012. He was with The Hong Kong University of Science and Technology (HKUST), as a Research Associate, in 2016. He has been with Chungbuk National University (CBNU), South Korea, since 2017. He has been an Associate Professor with the Department of Computer Engineering, CBNU, since 2021. His research interests include networking technologies, sensing and data analytics, and mobile and the Internet of Things based smart applications.

...